

auth.py

```
from flask import Blueprint, request, jsonify, make_response
from bson.objectid import ObjectId
import bcrypt
import jwt
import datetime
import globals
from decorators import jwt_required

auth_bp = Blueprint('auth_bp', __name__)

users = globals.db.users
blacklist = globals.db.blacklist

@auth_bp.route('/api/users/register', methods=['POST'])
def register():
    data = request.get_json()

    if not data or not data.get('username') or not data.get('password') or not data.get('email'):
        return make_response(jsonify({'message': 'Username, email, and password are required'}), 400)

    # for checking if username OR email is already used
    if users.find_one({'$or': [{'username': data['username']}, {'email': data['email']}]}):
```

```
    return make_response(jsonify({'message': 'Username or Email is already registered'}),
409)
```

```
salt = bcrypt.gensalt()
```

```
hashed_password = bcrypt.hashpw(data['password'].encode('utf-8'), salt)
```

```
new_user = {
```

```
    "username": data['username'],
```

```
    "email": data['email'],
```

```
    "password": hashed_password.decode('utf-8'),
```

```
    "role": data.get('role', 'user'),
```

```
    "contact_preference": data.get('contact_preference', 'email'),
```

```
    "is_verified": False,
```

```
    "created_at": datetime.datetime.utcnow()
```

```
}
```

```
result = users.insert_one(new_user)
```

```
verification_link = f"http://127.0.0.1:5001/api/users/verify/{str(result.inserted_id)}"
```

```
return make_response(jsonify({
```

```
    'message': f"Account created for {data['username']}! Please verify your email.",
```

```
    'verification_link': verification_link
```

```
}), 201)
```

```
@auth_bp.route('/api/users/verify/<user_id>', methods=['GET'])
```

```

def verify_email(user_id):
    try:
        result = users.update_one(
            {"_id": ObjectId(user_id)},
            {"$set": {"is_verified": True}}
        )
        if result.matched_count == 0:
            return make_response(jsonify({"message": "User not found"}), 404)

        return make_response(jsonify({"message": "Email successfully verified! You can now
log in."}), 200)
    except:
        return make_response(jsonify({"message": "Invalid verification link"}), 400)

@auth_bp.route('/api/login', methods=['POST'])
def login():
    auth = request.authorization

    if not auth or not auth.username or not auth.password:
        return make_response(jsonify({'message': 'Missing username or password'}), 401)

    user = users.find_one({'username': auth.username})

    if not user:
        return make_response(jsonify({'message': 'User not found'}), 404)

    # BLOCK LOGIN IF NOT VERIFIED
    if not user.get('is_verified', False):

```

```
    return make_response(jsonify({'message': 'Please verify your email before logging in.'}),
403)
```

```
if bcrypt.checkpw(auth.password.encode('utf-8'), user['password'].encode('utf-8')):
```

```
    token = jwt.encode(
```

```
        {
```

```
            'username': user['username'],
```

```
            'role': user['role'],
```

```
            'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
```

```
        },
```

```
        globals.SECRET_KEY,
```

```
        algorithm="HS256"
```

```
    )
```

```
    return make_response(jsonify({
```

```
        'message': 'Login successful!',
```

```
        'token': token,
```

```
        'username': user['username'],
```

```
        'role': user['role']
```

```
    })), 200)
```

```
    return make_response(jsonify({'message': 'Incorrect password'}), 401)
```

```
@auth_bp.route('/api/logout', methods=['GET'])
```

```
@jwt_required
```

```
def logout(current_user):
```

```
    token = None
```

```

if 'Authorization' in request.headers:
    token = request.headers['Authorization'].split(" ")[1]
elif 'x-access-token' in request.headers:
    token = request.headers['x-access-token']

blacklist.insert_one({'token': token})

return make_response(jsonify({'message': 'Logout successful'}), 200)

@auth_bp.route('/api/users', methods=['GET'])
@jwt_required
def get_all_users(current_user):
    if current_user.get('role') != 'admin':
        return make_response(jsonify({'message': 'Admin access required'}), 403)

    users_cursor = users.find({}, {"password": 0})
    users_list = []
    for user in users_cursor:
        user['_id'] = str(user['_id'])
        users_list.append(user)

    return make_response(jsonify({"count": len(users_list), "users": users_list}), 200)

```

farms.py

```

from flask import Blueprint, jsonify, request, make_response

```

```
import requests

from datetime import datetime

from bson.objectid import ObjectId


import globals

from decorators import jwt_required


farms_bp = Blueprint('farms_bp', __name__)

farms = globals.db.farms


def get_farm_if_authorised(farm_id, current_user):

    try:

        oid = ObjectId(farm_id)

    except:

        return None, make_response(jsonify({"message": "Invalid farm ID format"}), 400)


    farm = farms.find_one({"_id": oid})

    if not farm:

        return None, make_response(jsonify({"message": "Farm not found"}), 404)


    if current_user.get('role') == 'admin':

        return farm, None


    if str(farm['owner_id']) != str(current_user['_id']):

        return None, make_response(jsonify({"message": "Access denied. You can only manage your own farms."}), 403)
```

```
return farm, None
```

```
@farms_bp.route('/api/farms', methods=['GET'])
```

```
def get_all_farms():
```

```
    farms_cursor = farms.find({})
```

```
    farms_list = []
```

```
    for farm in farms_cursor:
```

```
        farm['_id'] = str(farm['_id'])
```

```
        farm['owner_id'] = str(farm['owner_id'])
```

```
        farms_list.append(farm)
```

```
    return make_response(jsonify(farms_list), 200)
```

```
@farms_bp.route('/api/farms/<farm_id>', methods=['GET'])
```

```
def get_single_farm(farm_id):
```

```
    try:
```

```
        farm = farms.find_one({"_id": ObjectId(farm_id)})
```

```
    except:
```

```
        return make_response(jsonify({"message": "Invalid farm ID format"}), 400)
```

```
    if not farm:
```

```
        return make_response(jsonify({"message": "Farm not found"}), 404)
```

```
    farm['_id'] = str(farm['_id'])
```

```
    farm['owner_id'] = str(farm['owner_id'])
```

```
    return make_response(jsonify(farm), 200)
```

```
@farms_bp.route('/api/farms', methods=['POST'])
@jwt_required
def create_farm(current_user):
    data = request.get_json()
    if not data or not data.get('farm_name'):
        return make_response(jsonify({"message": "Please provide at least a farm_name"}),
400)
```

```
    data['owner_id'] = current_user['_id']
    data.setdefault('sensors', [])
    data.setdefault('weather_logs', [])
    data.setdefault('alerts_history', [])
    data['created_at'] = datetime.now()

    result = farms.insert_one(data)

    return make_response(jsonify({"message": "Farm registered successfully!", "farm_id":
str(result.inserted_id)}), 201)
```

```
@farms_bp.route('/api/farms/<farm_id>', methods=['PUT'])
```

```
@jwt_required
```

```
def update_farm(current_user, farm_id):
    farm, err = get_farm_if_authorised(farm_id, current_user)
    if err: return err

    data = request.get_json()
    if not data:
```



```

        return make_response(jsonify({"message": "No update data provided"}), 400)

allowed_fields = ['farm_name', 'crop_type', 'address', 'location']
updates = {k: v for k, v in data.items() if k in allowed_fields}

if not updates:
    return make_response(jsonify({"message": f"No valid fields provided."}), 400)

farms.update_one({"_id": ObjectId(farm_id)}, {"$set": updates})
return make_response(jsonify({"message": "Farm updated successfully!"}), 200)

@farms_bp.route('/api/farms/<farm_id>', methods=['DELETE'])
@jwt_required
def delete_farm(current_user, farm_id):
    if current_user.get('role') != 'admin':
        return make_response(jsonify({"message": "Admin access required"}), 403)

    try:
        result = farms.delete_one({"_id": ObjectId(farm_id)})
    except:
        return make_response(jsonify({"message": "Invalid farm ID format"}), 400)

    if result.deleted_count == 0:
        return make_response(jsonify({"message": "Farm not found"}), 404)

    return make_response(jsonify({"message": "Farm deleted successfully"}), 200)

```

```

@farms_bp.route('/api/farms/<farm_id>/sensors', methods=['POST'])
@jwt_required
def add_sensor(current_user, farm_id):
    farm, err = get_farm_if_authorised(farm_id, current_user)
    if err: return err

    data = request.get_json()
    if not data or not data.get('sensor_id') or not data.get('type'):
        return make_response(jsonify({"message": "sensor_id and type required"}), 400)

    new_sensor = {
        "sensor_id": data['sensor_id'],
        "type": data['type'],
        "status": data.get('status', True),
        "readings": data.get('readings', [])
    }

    farms.update_one({"_id": ObjectId(farm_id)}, {"$push": {"sensors": new_sensor}})

    return make_response(jsonify({"message": "Sensor added to farm!", "sensor":
new_sensor}), 201)

@farms_bp.route('/api/farms/search', methods=['GET'])
def search_farms():
    search_term = request.args.get('q')
    if not search_term:

```

```
return make_response(jsonify({"message": "Provide search term using ?q="}), 400)
```

```
search_results = farms.find({"$text": {"$search": search_term}})
```

```
farms_list = []
```

```
for farm in search_results:
```

```
    farm['_id'] = str(farm['_id'])
```

```
    farm['owner_id'] = str(farm['owner_id'])
```

```
    farms_list.append(farm)
```

```
return make_response(jsonify({"results_count": len(farms_list), "data": farms_list}), 200)
```

```
@farms_bp.route('/api/farms/<farm_id>/sync_weather', methods=['POST'])
```

```
@jwt_required
```

```
def sync_weather(current_user, farm_id):
```

```
    farm, err = get_farm_if_authorised(farm_id, current_user)
```

```
    if err: return err
```

```
    lng = farm['location']['coordinates'][0]
```

```
    lat = farm['location']['coordinates'][1]
```

```
    weather_url = f"https://api.open-  
meteo.com/v1/forecast?latitude={lat}&longitude={lng}&current_weather=true"
```

```
    try:
```

```
        response = requests.get(weather_url)
```

```
        weather_data = response.json()
```

```
        current_weather = weather_data.get('current_weather', {})
```

```

new_log = {
    "timestamp": datetime.now(),
    "temperature_celsius": current_weather.get('temperature'),
    "windspeed": current_weather.get('windspeed'),
    "conditions": "Synced from Open-Meteo API"
}

farms.update_one({"_id": ObjectId(farm_id)}, {"$push": {"weather_logs": new_log}})
return make_response(jsonify({"message": "Weather synced!", "new_log": new_log}),
200)

except Exception as e:
    return make_response(jsonify({"message": "API error", "error": str(e)}), 500)

@farms_bp.route('/api/farms/alerts/broadcast', methods=['POST'])
@jwt_required
def broadcast_alert(current_user):
    if current_user.get('role') != 'admin':
        return make_response(jsonify({"message": "Admin access required"}), 403)

    data = request.get_json()
    if not data or 'alert_type' not in data or 'danger_zone' not in data:
        return make_response(jsonify({"message": "alert_type and danger_zone required"}),
400)

    geo_query = {"location": {"$geoWithin": {"$geometry": data['danger_zone']}}}

```

```
affected_farms = list(farms.find(geo_query))
```

```
alert_entry = {  
    "alert_type": data['alert_type'],  
    "timestamp": datetime.now(),  
    "message": f"EMERGENCY: {data['alert_type']} warning issued!",  
    "issued_by": current_user.get('username', 'admin')  
}
```

```
farms.update_many(geo_query, {"$push": {"alerts_history": alert_entry}})  
  
return make_response(jsonify({"message": "Alert broadcast!", "farms_notified":  
len(affected_farms)}), 200)
```

```
@farms_bp.route('/api/farms/<farm_id>/insights', methods=['GET'])
```

```
@jwt_required
```

```
def get_farm_insights(current_user, farm_id):
```

```
    farm, err = get_farm_if_authorised(farm_id, current_user)
```

```
    if err: return err
```

```
    try:
```

```
        pipeline = [  
            {"$match": {"_id": ObjectId(farm_id)}},  
            {"$unwind": "$weather_logs"},  
            {"$group": {  
                "_id": "$_id",  
                "farm_name": {"$first": "$farm_name"},
```

```

        "average_temp": {"$avg": "$weather_logs.temperature_celsius"},
        "average_wind": {"$avg": "$weather_logs.windspeed"}
    }}
]
result = list(farms.aggregate(pipeline))

if not result:
    return make_response(jsonify({"message": "Not enough data"}), 404)

insights = result[0]
insights['_id'] = str(insights['_id'])

if insights.get('average_temp'): insights['average_temp'] =
round(insights['average_temp'], 2)

if insights.get('average_wind'): insights['average_wind'] =
round(insights['average_wind'], 2)

return make_response(jsonify({"message": "Insights generated", "dashboard_data":
insights}), 200)

except Exception as e:
    return make_response(jsonify({"message": "Error", "error": str(e)}), 500)

@farms_bp.route('/api/farms/<farm_id>/irrigation_check', methods=['GET'])
@jwt_required
def check_irrigation(current_user, farm_id):
    farm, err = get_farm_if_authorised(farm_id, current_user)
    if err: return err

```

```

try:
    moisture_level = None
    for sensor in farm.get('sensors', []):
        if sensor.get('type') == 'Soil Moisture':
            readings = sensor.get('readings', [])
            if readings:
                moisture_level = readings[-1]['value']
            break

    if moisture_level is None:
        return make_response(jsonify({"message": "No soil moisture sensor found."}), 404)

    if moisture_level < 20.0:
        return make_response(jsonify({"status": "WARNING", "moisture": moisture_level}),
200)
    else:
        return make_response(jsonify({"status": "OK", "moisture": moisture_level}), 200)

except Exception as e:
    return make_response(jsonify({"message": "Error", "error": str(e)}), 500)

@farms_bp.route('/api/farms/region/<region_name>/insights', methods=['GET'])
def get_regional_insights(region_name):
    try:
        pipeline = [

```

```

        {"$match": {"address.area_name": region_name}},
        {"$unwind": "$weather_logs"},
        {"$group": {
            "_id": region_name,
            "community_avg_temp": {"$avg": "$weather_logs.temperature_celsius"},
            "unique_farms": {"$addToSet": "$_id"}
        }},
        {"$project": {
            "community_avg_temp": {"$round": ["$community_avg_temp", 2]},
            "total_farms_included": {"$size": "$unique_farms"}
        }}
    ]

    result = list(farms.aggregate(pipeline))

    if not result:

        return make_response(jsonify({"message": "No data found."}), 404)

    return make_response(jsonify({"message": f"Community averages", "data": result[0]}),
200)

except Exception as e:

    return make_response(jsonify({"message": "Error", "error": str(e)}), 500)

```

app.py

```

from flask import Flask, jsonify, make_response

```



```
from flask_cors import CORS

# import blueprints
from blueprints.auth.auth import auth_bp
from blueprints.farms.farms import farms_bp

app = Flask(__name__)
CORS(app)

#register blueprints
app.register_blueprint(auth_bp)
app.register_blueprint(farms_bp)

@app.route('/', methods=['GET'])
def home():
    return make_response(jsonify({"message": "Welcome to the Smart Agriculture API!"}),
200)

if __name__ == '__main__':
    app.run(debug=True, port=5001)
```

decorators.py

```
from flask import jsonify, request, make_response
import jwt
from functools import wraps
```

```
import globals
```

```
users = globals.db.users
```

```
blacklist = globals.db.blacklist
```

```
def jwt_required(f):
```

```
    @wraps(f)
```

```
    def decorated(*args, **kwargs):
```

```
        token = None
```

```
        # Accept token from authorization header or x-access-token
```

```
        if 'Authorization' in request.headers:
```

```
            token = request.headers['Authorization'].split(" ")[1]
```

```
        elif 'x-access-token' in request.headers:
```

```
            token = request.headers['x-access-token']
```

```
        if not token:
```

```
            return make_response(jsonify({'message': 'Token is missing! Please log in.'}), 401)
```

```
        try:
```

```
            data = jwt.decode(token, globals.SECRET_KEY, algorithms=["HS256"])
```

```
            current_user = users.find_one({"username": data['username']})
```

```
        except:
```

```
            return make_response(jsonify({'message': 'Token is invalid or expired!'}), 401)
```

```
        # Check if the user logged out
```

```
bl_token = blacklist.find_one({'token': token})

if bl_token is not None:

    return make_response(jsonify({'message': 'Token has been cancelled/logged out'}),
401)


return f(current_user, *args, **kwargs)

return decorated
```

globals.py

```
from pymongo import MongoClient

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.smart_agri_db

SECRET_KEY = 'dream_project'
```

make_dataset.py

```
from pymongo import MongoClient
import random
from datetime import datetime, timedelta
import bcrypt

client = MongoClient("mongodb://127.0.0.1:27017")
db = client.smart_agri_db
```

```
db.farms.drop()
```

```
db.users.drop()
```

```
print("Generating Smart Agriculture Dataset...")
```

```
# 1.USERS
```

```
# One admin account and two regular farmer accounts
```

```
# All use "password123"
```

```
users_data = []
```

```
usernames = ["admin_user", "farmer_john", "farmer_mary"]
```

```
roles = ["admin", "user", "user"]
```

```
for i in range(3):
```

```
    salt = bcrypt.gensalt()
```

```
    hashed_password = bcrypt.hashpw(b"password123", salt)
```

```
    users_data.append({
```

```
        "username": usernames[i],
```

```
        "email": f"{usernames[i]}@smartagri.com",
```

```
        "password": hashed_password.decode('utf-8'),
```

```
        "role": roles[i],
```

```
        "contact_preference": "email",
```

```
        "is_verified": True,
```

```
        "created_at": datetime.now()
```

```
})
```

```
inserted_users = db.users.insert_many(users_data)
```

```
user_ids = inserted_users.inserted_ids
```

```
print(f" Created {len(user_ids)} users.")
```

```
# 2.FARMS
```

```
# 50 farm Northern Ireland
```

```
# Each farm has sensors with historical readings,
```

```
# weather logs, and a blank alerts history to start
```

```
crop_types = ["Wheat", "Corn", "Vineyard", "Soybeans", "Potatoes"]
```

```
areas      = ["Belfast", "Derry", "Lisburn", "Newry", "Armagh"]
```

```
postcodes = ["BT7 1NN", "BT48 7NL", "BT28 1AB", "BT35 6PB", "BT60 1NT"]
```

```
# coordinates
```

```
base_lat = 54.5
```

```
base_lng = -6.5
```

```
farms_data = []
```

```
for i in range(50):
```

```
    # each farm has two farmer accounts (not admin)
```

```
    owner_id = random.choice(user_ids[1:])
```

```
    area_index = random.randint(0, 4)
```

```

base_date = datetime(2025, 1, 1) + timedelta(days=random.randint(0, 60))

# add a set of IoT sensors for this farm
sensors = []

for j in range(random.randint(2, 5)):
    sensor_type = random.choice(["Soil Moisture", "Temperature", "pH Level"])

    # Each sensor has a week of historical readings
    readings = []

    for k in range(7):
        reading_time = base_date + timedelta(days=k)
        value = round(random.uniform(10.0, 60.0), 2)
        readings.append({
            "timestamp": reading_time,
            "value": value
        })

    sensors.append({
        "sensor_id": f"SEN-{i}-{j}",
        "type": sensor_type,
        "status": random.choice([True, True, False]),
        "readings": readings
    })

```

```

# 3 historical weather log entries per farm
weather_logs = []
for w in range(3):
    log_time = base_date + timedelta(days=w)
    weather_logs.append({
        "timestamp":    log_time,
        "temperature_celsius": round(random.uniform(5.0, 25.0), 1),
        "windspeed":    round(random.uniform(5.0, 30.0), 1),
        "humidity_percent": random.randint(40, 95),
        "conditions":    random.choice(["Clear", "Rain", "Cloudy", "Overcast"])
    })

farm_document = {
    "farm_name": f"Farm Plot {i + 1}",
    "owner_id": owner_id,
    "crop_type": random.choice(crop_types),
    "address": {
        "area_name": areas[area_index],
        "postcode": postcodes[area_index]
    },
    "location": {
        "type": "Point",
        # coordinates slightly around Northern Ireland
        "coordinates": [
            round(base_lng + random.uniform(-1.0, 1.0), 4),
            round(base_lat + random.uniform(-0.5, 0.5), 4)
        ]
    }
}

```

```

        ]
    },
    "sensors": sensors,
    "weather_logs": weather_logs,
    "alerts_history": []
}

farms_data.append(farm_document)

db.farms.insert_many(farms_data)

print(f" Created 50 farms with sensors and weather logs.")


# 3.CREATE MONGODB INDEXES

#2dsphere index for the geospatial queries

# Geospatial index

db.farms.create_index([("location", "2dsphere")])

# text index

db.farms.create_index([
    ("address.area_name", "text"),
    ("address.postcode", "text")
])

print(" Indexes created (2dsphere + compound text).")

print("\nDone! Login credentials for testing:")

```



```
print(" Admin: username=admin_user / password=password123")  
print(" Farmer: username=farmer_john / password=password123")  
print(" Farmer: username=farmer_mary / password=password123")
```